OpenClaw 企业版管理员运维手册

> 面向已部署 OpenClaw on Amazon Bedrock AgentCore 方案（钉钉频道）的企业管理员
>
> 本文档涵盖平台日常运维、用户管理、安全巡检、版本升级等管理员所需的全部操作指南。方案交付后，后续的运维管理由企业管理员自行负责。

项目	内容
目标受众	企业 IT 管理员、安全运维人员、平台负责人
前置条件	已完成 OpenClaw 部署及钉钉机器人接入

一、平台运维与用户管理

模块 1：架构回顾与关键组件

1.1 整体架构

```
钉钉云 <==WebSocket==> ECS Fargate (dingtalk-bridge)
                            |
                                 +-- DynamoDB (用户身份解析)
                                 +-- AgentCore Runtime (AI 对话，每用户独立微虚拟机)
                                 +-- S3 (用户文件、工作区、图片)
                                 +-- Bedrock (大模型推理)
                            +-- 钉钉 REST API (回复消息)
```

1.2 核心组件说明

组件	职责	管理员关注点
ECS Fargate (钉钉桥接)	维持钉钉 WebSocket 长连接，收发消息	服务状态、日志、重启
AgentCore Runtime	每用户独立微虚拟机，运行 AI 对话	会话管理、镜像更新
DynamoDB 身份表	用户身份、白名单、会话、定时任务	用户管理、白名单
S3 存储	用户文件、工作区持久化、图片上传	数据生命周期、清理
Secrets Manager	钉钉凭证、系统密钥、用户 API 密钥	凭证轮换、安全
Bedrock Guardrails	内容过滤、PII 脱敏、话题拒绝	安全策略调整
CloudWatch	监控仪表板、告警、日志	日常巡检、异常排查

模块 2：用户管理

2.1 用户注册模式

模式	配置	适用场景
白名单模式（默认）	`registration_open: false`	企业内部精确控制
开放注册	`registration_open: true`	大量用户快速上线

> **企业建议**：生产环境保持白名单模式，确保只有授权人员可访问。

2.2 白名单管理操作

```
# 添加单个用户
./scripts/manage-allowlist.sh add dingtalk:用户ID

# 批量添加（从文件导入）
while read -r uid; do
  ./scripts/manage-allowlist.sh add "dingtalk:$uid"
done < dingtalk_users.txt

# 查看当前白名单
./scripts/manage-allowlist.sh list

# 移除用户
./scripts/manage-allowlist.sh remove dingtalk:用户ID
```

2.3 新用户入职流程

1. 新员工通过钉钉给机器人发送任意消息
2. 机器人回复拒绝消息，包含用户 ID（如 `dingtalk:01455368144039922107`）
3. 员工将 ID 发给管理员
4. 管理员执行 `./scripts/manage-allowlist.sh add dingtalk:01455368144039922107`
5. 员工再次发消息即可正常使用

2.4 员工离职处理

```
# 1. 从白名单移除
./scripts/manage-allowlist.sh remove dingtalk:用户ID

# 2. 检查并终止活跃会话（可选）
aws bedrock-agentcore stop-runtime-session \
  --agent-runtime-arn "<RUNTIME_ARN>" \
  --runtime-session-id "<SESSION_ID>" \
  --region us-west-2

# 3. 清理用户数据（如需要）
# 用户文件存储在 S3: s3://<BUCKET>/dingtalk_<用户ID>/
# 定时任务存储在 DynamoDB: CRON# 记录
# API 密钥存储在 Secrets Manager: openclaw/user/dingtalk_<用户ID>/*
```

2.5 跨频道绑定

如果企业同时使用钉钉和其他频道（如 Slack），用户可以通过”绑定账号”功能关联多个频道身份，共享同一会话和数据。绑定码有效期 10 分钟，一次性使用。

模块 3：日常运维操作

3.1 服务状态检查

```
# 检查钉钉桥接服务状态
aws ecs describe-services --cluster openclaw-dingtalk \
  --services openclaw-dingtalk-bridge --region us-west-2 \
  --query 'services[0].
{status:status,running:runningCount,desired:desiredCount}'

# 检查 AgentCore Runtime 状态
agentcore status --agent openclaw_agent --verbose
```

3.2 日志查看

```
# 钉钉桥接服务日志（实时跟踪）
aws logs tail /openclaw/dingtalk-bridge --follow --region us-
west-2

# Router Lambda 错误日志（最近 5 分钟）
aws logs filter-log-events \
  --log-group-name /openclaw/lambda/router --region us-west-2 \
  --start-time $(python3 -c "import time;
print(int((time.time()-300)*1000))") \
  --filter-pattern "ERROR" \
  --query 'events[*].message' --output text

# AgentCore 容器日志
agentcore invoke '{"action":"status"}' -a openclaw_agent
```

3.3 会话管理

```
# 查看用户会话信息
aws dynamodb query --table-name openclaw-identity --region us-
west-2 \
```

```
  --key-condition-expression "PK = :pk AND SK = :sk" \
  --expression-attribute-values '{":pk":
{"S":"USER#<internalUserId>"},":sk":{"S":"SESSION"}}'

# 强制终止用户会话（更新镜像后需要）
agentcore stop-session -a openclaw_agent -s <SESSION_ID>
```

3.4 常见故障排查

故障现象	排查步骤
机器人无响应	检查 ECS 服务状态 → 查看钉钉桥接日志 → 检查 Secrets Manager 凭证
响应缓慢	查看 CloudWatch `OpenClaw-Operations` 仪表板 → 检查 Bedrock 延迟指标
用户发消息被拒绝	确认白名单中是否有该用户 → 检查用户 ID 格式是否正确
会话突然断开	检查空闲超时（默认 30 分钟）→ 查看容器日志是否有 OOM 或异常退出

模块 4：镜像更新与部署

4.1 部署架构（混合部署）

组件	管理方式
基础设施（VPC、Lambda、DynamoDB 等）	CDK（`cdk deploy`）
AgentCore Runtime（容器镜像、运行时）	Starter Toolkit（`agentcore deploy`）

4.2 使用部署脚本

```
# 全量部署（3 阶段）
./scripts/deploy.sh

# 仅更新 Runtime
./scripts/deploy.sh --runtime-only
```

```
# 仅更新 CDK 基础设施
./scripts/deploy.sh --phase1
./scripts/deploy.sh --phase3
```

二、版本管理、多实例与日志排查

模块 5：日志体系与故障排查

5.1 日志架构全景

```
用户消息 → 钉钉桥接日志 → AgentCore 容器日志 → Bedrock 调用日志
            ↓                ↓                  ↓
     /openclaw/dingtalk-bridge  /openclaw/container    /aws/bedrock/invocation-logs
                                                        ↓
                                                  Token 处理 Lambda 日志
```

日志源	CloudWatch 日志组	包含内容
钉钉桥接服务	`/openclaw/dingtalk-bridge`	消息接收、用户解析、AgentCore 调用、回复发送
Router Lambda	`/openclaw/lambda/router`	Webhook 请求、身份解析、异步分发（其他频道使用）
Cron Lambda	`/openclaw/lambda/cron`	定时任务执行、会话预热、响应投递
API Gateway 访问日志	`/openclaw/api-access`	HTTP 请求路径、状态码、延迟
Bedrock 调用日志	`/aws/bedrock/invocation-logs`	模型输入/输出、Token 用量、Guardrail 结果
AgentCore 容器	`/openclaw/container`	容器启动、proxy 状态、

日志源	CloudWatch 日志组	包含内容
		OpenClaw 状态、工具调用
ECS 服务事件	AWS 控制台 ECS 页面	任务启停、部署状态、健康检查

5.2 钉钉桥接服务日志

钉钉桥接运行在 ECS Fargate 上，日志通过 CloudWatch 采集：

```
# 实时跟踪日志（最常用）
aws logs tail /openclaw/dingtalk-bridge --follow --region us-west-2

# 查看最近 30 分钟的日志
aws logs tail /openclaw/dingtalk-bridge --since 30m --region us-west-2

# 搜索错误日志
aws logs filter-log-events \
  --log-group-name /openclaw/dingtalk-bridge --region us-west-2 \
  --start-time $(python3 -c "import time; print(int((time.time()-3600)*1000))") \
  --filter-pattern "ERROR" \
  --query 'events[*].message' --output text

# 搜索特定用户的消息处理日志
aws logs filter-log-events \
  --log-group-name /openclaw/dingtalk-bridge --region us-west-2 \
  --start-time $(python3 -c "import time; print(int((time.time()-3600)*1000))") \
  --filter-pattern '"dingtalk:01455368144039922107"' \
  --query 'events[*].message' --output text
```

关键日志模式：

日志关键字	含义
`Received message from`	收到用户消息
`Resolved user / Created new user`	用户身份解析成功
`Invoking AgentCore session`	开始调用 AI 会话

日志关键字	含义
`AgentCore response`	收到 AI 回复
`Sent reply to`	回复已发送到钉钉
`ERROR`	错误（需关注）
`Registration denied`	用户不在白名单中
`Cold start` / `Session created`	新会话创建（首次或超时后）
`WebSocket disconnected` / `Reconnecting`	钉钉连接断开/重连

5.3 AgentCore 容器日志

容器内部的 `console.log/warn/error` 通过 `cloudwatch-logger.js` 自动上传到 CloudWatch 日志组 `/openclaw/container`，每个用户会话创建独立的日志流（格式：`{namespace}-{timestamp}`）：

```
# 查看容器日志（实时跟踪）
aws logs tail /openclaw/container --follow --region us-west-2

# 搜索特定用户的容器日志
aws logs filter-log-events \
  --log-group-name /openclaw/container --region us-west-2 \
  --start-time $(python3 -c "import time;
print(int((time.time()-3600)*1000))") \
  --log-stream-name-prefix "dingtalk_01455368144039922107" \
  --query 'events[*].message' --output text

# 搜索容器错误日志
aws logs filter-log-events \
  --log-group-name /openclaw/container --region us-west-2 \
  --start-time $(python3 -c "import time;
print(int((time.time()-3600)*1000))") \
  --filter-pattern "[ERROR]" \
  --query 'events[*].message' --output text
```

也可以通过 `invoke` 接口快速查看容器当前状态（不需要查日志）：

```
# 查看容器状态（不触发初始化）
agentcore invoke '{"action":"status"}' -a openclaw_agent
# 返回: {"openclawReady":true,"proxyReady":true,"uptime":
3600,"buildVersion":"v40",...}
```

5.4 Bedrock 调用日志查询

使用 CloudWatch Logs Insights 进行高级查询：

```
# 查询最近 1 小时的模型调用
aws logs start-query \
  --log-group-name /aws/bedrock/invocation-logs \
  --start-time $(date -d '1 hour ago' +%s) \
  --end-time $(date +%s) \
  --query-string 'fields @timestamp, modelId, inputTokenCount, outputTokenCount | sort @timestamp desc | limit 20' \
  --region us-west-2

# 获取查询结果（需要等几秒）
aws logs get-query-results --query-id <QUERY_ID> --region us-west-2

# 查询被 Guardrail 拦截的请求
aws logs start-query \
  --log-group-name /aws/bedrock/invocation-logs \
  --start-time $(date -d '24 hours ago' +%s) \
  --end-time $(date +%s) \
  --query-string 'filter guardrailAction = "GUARDRAIL_INTERVENED" | fields @timestamp, guardrailAction | sort @timestamp desc | limit 50' \
  --region us-west-2
```

5.5 常见问题排查流程图

```
用户报告"机器人没反应"
    │
    ├──→ 检查钉钉桥接服务状态
    │    aws ecs describe-services --cluster openclaw-dingtalk ...
    │    ├── 服务未运行 → 检查 ECS 事件、重启服务
    │    └── 服务运行中 ↓
    │
    ├──→ 检查钉钉桥接日志
    │    aws logs tail /openclaw/dingtalk-bridge --since 10m ...
    │    ├─ "Registration denied" → 用户不在白名单，添加白名单
    │    ├─ "WebSocket disconnected" → 检查钉钉凭证是否有效
    │    ├─ "ERROR.*AgentCore" → AgentCore 调用失败 ↓
    │    └── 无相关日志 → 钉钉连接问题，重启服务
```

```
    │
    ├──→ 检查 AgentCore 状态
    │    agentcore status --agent openclaw_agent --verbose
    │    agentcore invoke '{"action":"status"}' -a openclaw_agent
    │    ├─ Runtime 异常 → 检查 Runtime 配置、环境变量
    │    └── 容器正常 → 检查 Bedrock 调用日志
    │
    └──→ 检查 Bedrock 调用日志
         ├── 限流 (ThrottlingException) → 申请提升配额
         ├── 模型错误 → 检查 model_id 配置
         └─ Guardrail 拦截 → 调整 Guardrails 策略

用户报告"回复内容被截断"
    │
    ├──→ 检查钉钉消息长度限制（单条 20,000 字符）
    │    ├── 超长回复 → 钉钉桥接自动分段发送，检查是否所有段都发出
    │    └── 未超长 ↓
    │
    └──→ 检查 Bedrock 响应
         ├── 输出 token 达到上限 → 模型输出被截断，属正常行为
         └─ Guardrail 截断 → 查看拦截日志

用户报告"响应很慢"
    │
    ├──→ 是否是冷启动？（新用户首次消息 / 30 分钟无活动后）
    │    ├── 是 → 正常，AgentCore 微虚拟机创建需 30-60 秒
    │    └── 否 ↓
    │
    ├──→ 检查 CloudWatch Operations 仪表板
    │    ├─ Bedrock 延迟高 → 检查模型服务状态
    │    └── 延迟正常 → 检查是否使用了 deep-research-pro 等耗时技能
    │
    └──→ 检查网络（VPC 端点、NAT）
```

5.6 ECS 服务事件与部署日志

```
# 查看 ECS 服务最近事件（部署、健康检查）
aws ecs describe-services --cluster openclaw-dingtalk \
  --services openclaw-dingtalk-bridge --region us-west-2 \
  --query 'services[0].events[:10].{time:createdAt,msg:message}' --
output table
```

```
        # 查看当前运行任务详情
        aws ecs list-tasks --cluster openclaw-dingtalk --service-name
openclaw-dingtalk-bridge --region us-west-2
        aws ecs describe-tasks --cluster openclaw-dingtalk --tasks
<TASK_ARN> --region us-west-2 \
          --query 'tasks[0].
{status:lastStatus,health:healthStatus,startedAt:startedAt,stoppedReason:stoppedRe
```

模块 6：创建多个钉钉机器人

6.1 为什么需要多个钉钉机器人

场景	说明
部门隔离	不同部门使用不同机器人，独立的白名单和用量统计
功能差异化	研发机器人启用浏览器和代码技能，业务机器人只启用基础功能
测试/生产分离	测试机器人用于验证新版本，生产机器人服务正式用户
合规要求	不同安全等级的数据通过不同机器人处理

6.2 当前架构：单机器人

```
钉钉机器人 A (唯一)
    |
    v
ECS Fargate (openclaw-dingtalk-bridge, 1 个服务)
    |
    v
AgentCore Runtime (openclaw_agent, 共享)
```

所有钉钉用户通过同一个机器人入口，共享同一套基础设施。用户间通过 AgentCore 微虚拟机隔离数据。

6.3 多机器人方案（设计思路）

> **注意**：当前方案默认支持单个钉钉机器人。多机器人能力尚未内置，需要根据企业实际场景进行定制化开发。以下提供设计思路供参考。

整体思路

```
钉钉机器人 A (研发部)            钉钉机器人 B (业务部)
    |                               |
    v                               v
CloudFormation 栈 A             CloudFormation 栈 B
(独立 ECS 服务 + 日志组)      (独立 ECS 服务 + 日志组)
    |                               |
    v                               v
AgentCore Runtime A             AgentCore Runtime B (可选，或共用 A)
```

每个钉钉机器人需要以下独立资源：

资源	说明
钉钉开放平台应用	独立的 AppKey / AppSecret
Secrets Manager 密钥	存储各自的凭证（如 `openclaw/channels/dingtalk-biz`）
ECS Fargate 服务	维持各自的钉钉 WebSocket 长连接
CloudWatch 日志组	独立的日志流，便于分别排查
AgentCore Runtime（可选）	如需不同模型/版本，部署独立 Runtime；否则可共用

推荐实现路径：独立 CloudFormation 栈

推荐将每个钉钉机器人部署为独立的 CloudFormation 栈，优势：

- **独立生命周期**：每个栈可单独部署、回滚、删除，互不影响
- **基础设施即代码**：CloudFormation 自动管理 ECS 集群、任务定义、IAM 角色等，可重复、可审计
- **资源隔离清晰**：通过栈名区分资源（如 `openclaw-dingtalk-dev` vs `openclaw-dingtalk-biz`）

实现时需要对现有 `DingTalkStack`（`stacks/dingtalk_stack.py`）进行参数化改造：

- 添加 `instance_name` 参数，使集群名、服务名、日志组名等可定制
- 在 `app.py` 中为每个机器人实例化独立的栈
- 每个栈引用各自的 Secrets Manager 密钥

需要关注的设计问题

问题	说明
用户身份隔离	同一钉钉用户在不同机器人中 `staffId` 相同。默认情况下共享同一 AgentCore 会话和 S3 工作区。如需完全隔离，需修改桥接代码中的 `actorId` 构建逻辑（如加入机器人标识前缀 `dingtalk-biz:<staffId>`）
白名单管理	共用 DynamoDB 身份表。如需按机器人分别控制准入，需扩展白名单机制
Runtime 共用 vs 独立	共用 Runtime 成本低但功能一致；独立 Runtime 可使用不同模型/版本，但管理复杂度和成本翻倍
VPC 端点	多个 ECS 服务共用同一 VPC 和 VPC 端点，无需额外网络配置
成本	每增加一个 ECS Fargate 服务约 $30-50/月

建议

1. **明确需求**：先确定多机器人的核心诉求（部门隔离？功能差异？测试/生产分离？），不同目标对应不同的定制化深度
2. **联系方案提供方**：多机器人属于定制化开发范围，建议与方案提供方沟通具体需求，评估改造工作量
3. **渐进式实施**：建议先用单机器人 + 白名单管理覆盖大部分场景，确认确实需要多机器人后再投入开发

模块 7：上游仓库更新与版本升级

7.1 版本更新策略

策略	适用场景	风险等级
直接合并	小版本更新、安全补丁	低
分支测试后合并	功能更新、依赖升级	中

策略	适用场景	风险等级
蓝绿部署	大版本升级、架构变更	高

> **企业建议**：所有更新先在测试环境验证，确认无回归后再推生产。

7.2 拉取上游更新

```
# 1. 添加上游仓库（首次）
git remote add upstream <上游仓库地址>

# 2. 获取上游最新代码
git fetch upstream main

# 3. 查看上游变更内容
git log --oneline HEAD..upstream/main
git diff HEAD..upstream/main --stat

# 4. 重点关注以下文件的变更
#    - bridge/      (容器代码 — 影响运行时行为)
#    - stacks/      (CDK 基础设施 — 影响 AWS 资源)
#    - cdk.json     (配置参数 — 可能新增必填项)
#    - lambda/      (Lambda 函数 — 影响消息路由)
#    - bridge/skills/ (技能脚本 — 影响 AI 功能)
```

7.3 变更评估清单

在合并前，逐项检查：

检查项	命令 / 方法	关注点
CDK 资源变更	`git diff HEAD..upstream/main -- stacks/`	新增/删除的 AWS 资源，IAM 权限变化
cdk.json 新参数	`git diff HEAD..upstream/main -- cdk.json`	新增配置项是否需要填写
容器代码变更	`git diff HEAD..upstream/main -- bridge/`	启动逻辑、安全控制、工具列表变化
依赖更新	`git diff HEAD..upstream/main -- bridge/package.json requirements.txt`	新增依赖、版本跳跃

检查项	命令 / 方法	关注点
Lambda 变更	`git diff HEAD..upstream/main -- lambda/`	路由逻辑、频道处理、图片处理
环境变量	搜索新增 `process.env.` 和 `os.environ`	是否需要在 Runtime 更新中传入新环境变量
破坏性变更	查看 CHANGELOG / release notes	是否有不兼容的 API 或配置变更

7.4 安全合并与部署

```
# 1. 在本地分支上合并（推荐 rebase 方式保持历史清晰）
git checkout main
git rebase upstream/main
# 或使用 merge: git merge upstream/main

# 2. 解决冲突（如有）
#    重点关注 cdk.json — 保留本地配置（account、runtime_id 等）

# 3. 运行测试确认无回归
cd bridge && node --test lightweight-agent.test.js
cd bridge && node --test image-support.test.js
cd bridge && node --test scoped-credentials.test.js
cd lambda/router && python -m pytest -v

# 4. CDK 合规检查
source .venv/bin/activate
cdk synth 2>&1 | grep -i "error\|warning"
cdk diff  # 预览基础设施变更

# 5. 分阶段部署
./scripts/deploy.sh --phase1          # 先部署基础设施
./scripts/deploy.sh --runtime-only     # 再更新容器镜像
./scripts/deploy.sh --phase3          # 最后部署依赖栈

# 6. 终止旧会话，使新版本生效
agentcore stop-session -a openclaw_agent -s <SESSION_ID>

# 7. 验证新版本
```

```
agentcore invoke '{"action":"status"}' -a openclaw_agent
# 检查 BUILD_VERSION 是否更新
```

7.5 回滚方案

```
# 方案 A：回滚容器镜像（不影响基础设施）
# 查看 ECR 中的历史镜像
aws ecr describe-images --repository-name bedrock-agentcore-openclaw_agent \
  --region us-west-2 \
  --query 'sort_by(imageDetails,&imagePushedAt)[*].{tag:imageTags[0],pushed:imagePushedAt}' \
  --output table

# 更新 Runtime 指向旧镜像（注意：必须包含完整环境变量！）
aws bedrock-agentcore-control update-agent-runtime \
  --agent-runtime-id <RUNTIME_ID> \
  --agent-runtime-artifact '{"containerConfiguration":{"containerUri":"<ECR_URI>:<旧版本TAG>"}}' \
  --environment-variables '{...完整环境变量...}' \
  --region us-west-2

# 方案 B：回滚代码 + 基础设施
git revert HEAD   # 或 git reset --hard <上次稳定版本>
./scripts/deploy.sh
```

7.6 自动化更新检查（可选）

建议在企业内部 CI/CD 流程中加入：

```
定期检查上游 → 自动 diff 报告 → 人工评审 → 测试环境验证 → 生产部署
```

可通过 GitHub Actions / GitLab CI 设置定时 `git fetch upstream` 并生成变更报告通知管理员。

模块 8：多版本 / 多实例管理

8.1 架构概述

当前架构为**单 Runtime 实例**：所有钉钉用户共享同一个 AgentCore Runtime（同一容器镜像、同一 AI 模型、同一 Guardrails 策略）。不同用户通过 **per-user 微虚拟机** 实现数据隔离，但功能和版本相同。

如果企业需要为不同部门或用户群体提供差异化服务，有以下方案：

8.2 方案对比

方案	差异化维度	复杂度	成本影响	适用场景
A. 单实例 + 用户级配置	AI 模型、技能、Guardrails	低	无额外成本	轻度差异化
B. 多 Runtime 实例	容器镜像、环境变量、全部配置	中	按实例数线性增长	完全独立的服务版本
C. 多 CDK 部署（多环境）	完全独立的基础设施	高	整套基础设施翻倍	测试/生产环境隔离

8.3 方案 A：单实例 + 用户级配置（推荐起步方案）

在当前单 Runtime 架构下，通过以下机制实现轻度差异化：

A1. 用户级 AI 模型切换

当前不直接支持按用户切换模型，但可通过以下方式间接实现：

- **方式一**：修改 `agentcore-proxy.js`，根据请求中的 `userId` 路由到不同的 Bedrock 模型
- **方式二**：部署多个代理配置文件，让不同用户组使用不同的 `openclaw.json` 配置

A2. 用户级技能差异化

通过 `clawhub-manage` 技能，用户可自行安装/卸载技能。管理员可以：

```
# 为特定用户预安装技能（通过 S3 工作区预置）
# 1. 将预配置的 .openclaw/ 目录上传到用户的 S3 命名空间
aws s3 sync ./preset-workspace/ \
  s3://openclaw-user-files-<ACCOUNT>-us-west-2/dingtalk_<用户ID>/.openclaw/ \
  --region us-west-2
```

A3. 白名单分组

虽然当前白名单是扁平结构，但可以通过命名约定来管理用户分组：

```
# 按部门添加用户时做好记录
# 建议维护一个 CSV 文件记录用户分组
# dingtalk_users.csv:
# 用户ID,部门,角色,添加日期
# 01455368144039922107,研发部,开发者,2026-03-25
# 06283719502847261033,市场部,普通用户,2026-03-25
```

8.4 方案 B：多 Runtime 实例（完全版本隔离）

为不同用户群体部署独立的 AgentCore Runtime 实例，每个实例可以有不同的容器镜像、AI 模型和环境配置。

架构图

```
钉钉用户群体 A（研发部）          钉钉用户群体 B（业务部）
       |                          |
       v                          v
  ECS Service A / B（路由分发）
       |                          |
       v                          v
  AgentCore Runtime A              AgentCore Runtime B
  - 镜像: v2.1 (最新功能)          - 镜像: v2.0 (稳定版)
  - 模型: Claude Opus 4.6          - 模型: Claude Sonnet 4.6
  - Guardrails: 宽松策略           - Guardrails: 严格策略
  - 技能: 全部启用                 - 技能: 基础技能
```

B1. 创建第二个 Runtime 实例

```
# 1. 配置第二个 Agent（不同名称）
agentcore configure --name openclaw_agent_biz \
```

```
  --entrypoint bridge/agentcore-contract.js \
  --execution-role <ROLE_ARN> --region us-west-2 --vpc \
  --subnets <SUBNET_IDS> --security-groups <SG_ID> \
  --deployment-type container --language typescript --non-interactive

# 2. 使用不同的镜像标签和环境变量部署
agentcore deploy --agent openclaw_agent_biz --local-build \
  --auto-update-on-conflict \
  --env "BEDROCK_MODEL_ID=global.anthropic.claude-sonnet-4-6-v1" \
  --env "S3_USER_FILES_BUCKET=openclaw-user-files-..." \
  ...其他环境变量...
```

B2. 修改钉钉桥接指向不同 Runtime

每个 ECS 服务（钉钉机器人）通过环境变量 `AGENTCORE_RUNTIME_ARN` 指向不同的 Runtime。参考模块 6 创建多个钉钉机器人的方法，为每个机器人的 ECS 服务配置不同的 Runtime ARN：

```
# 机器人 A → Runtime A
AGENTCORE_RUNTIME_ARN=arn:aws:bedrock-agentcore:us-west-2:<ACCOUNT>:runtime/<RUNTIME_A>

# 机器人 B → Runtime B
AGENTCORE_RUNTIME_ARN=arn:aws:bedrock-agentcore:us-west-2:<ACCOUNT>:runtime/<RUNTIME_B>
```

B3. DynamoDB 用户分组扩展

在身份表中为用户添加分组属性：

```
# 为用户设置分组（添加 group 属性到 USER# 记录）
aws dynamodb update-item \
  --table-name openclaw-identity \
  --key '{"PK":{"S":"USER#<internalUserId>"},"SK":{"S":"PROFILE"}}' \
  --update-expression "SET #g = :g" \
  --expression-attribute-names '{"#g":"group"}' \
  --expression-attribute-values '{":g":{"S":"biz_group"}}' \
  --region us-west-2
```

B4. 多实例管理成本

组件	每增加一个 Runtime 实例
AgentCore Runtime	按活跃会话计费（独立计算）
ECR 镜像	共用仓库，不同标签，存储成本可忽略
NAT / VPC	共用现有 VPC，无额外成本
DynamoDB	共用身份表，增加路由字段
CloudWatch	需要额外的仪表板/告警配置

8.5 方案 C：多环境部署（测试/灰度/生产）

通过 CDK 的 `account` 和 `region` 参数，或使用不同的资源前缀，部署完全独立的基础设施。

C1. 推荐的环境拓扑

环境	用途	钉钉机器人	用户范围
测试环境	功能验证、安全测试	测试机器人 A	管理员 + QA
灰度环境	新版本小范围验证	灰度机器人 B	种子用户
生产环境	全量用户服务	生产机器人 C	全部用户

C2. 多环境 CDK 配置

为每个环境创建独立的配置文件：

```
# 复制项目目录
cp -r sample-host-openclaw-on-amazon-bedrock-agentcore openclaw-staging

# 修改 cdk.json 中的关键参数
cd openclaw-staging
# 修改 runtime_id、runtime_endpoint_id（部署后自动填充）
# 修改 default_model_id（可选：灰度环境用更便宜的模型）
# 在钉钉开放平台创建独立的测试/灰度机器人应用
```

C3. 灰度发布流程

1. 新版本代码合并到 `main` 分支
2. 在测试环境部署并验证

```
./scripts/deploy.sh  # 在测试环境项目目录执行
```

3. 运行自动化测试

```
cd tests/e2e && python -m pytest bot_test.py -v
```

4. 在灰度环境部署，通知种子用户测试
5. 观察 1-3 天，确认无问题
6. 在生产环境部署
7. 分批终止旧会话，让用户逐步切换到新版本

8.6 方案选择建议

企业规模	推荐方案	理由
< 50 人	**A. 单实例**	管理简单，成本最低
50-500 人	**A + C（测试/生产双环境）**	保证稳定性，支持安全验证
> 500 人或跨部门差异化需求	**B + C**	多实例隔离 + 多环境保障

三、安全管理与定期巡检

模块 9：安全架构深入

9.1 十层纵深防御

层级	安全控制	企业管理员职责
网络层	VPC 私有子网、VPC 端点、安全组	审查安全组规则，确认无多余入站规则
API 网关	速率限制、显式路由、访问日志	监控速率限制告警
身份认证	钉钉凭证验证、白名单、Cognito	管理白名单，轮换凭证
用户隔离	每用户微虚拟机、STS 作用域凭证	理解隔离机制，无需日常操作
加密	KMS CMK、传输加密、S3 版本控制	确认密钥自动轮换已启用
密钥管理	Secrets Manager、凭证环境变量隔离	定期轮换系统密钥

层级	安全控制	企业管理员职责
应用安全	SSRF 防护、路径遍历防护、输入校验	关注 CVE 扫描结果
内容安全	Bedrock Guardrails 内容过滤	调整过滤策略
容器安全	ARM64 精简镜像、ECR 扫描、工具拒绝列表	审查扫描结果
审计监控	CloudTrail、CloudWatch、异常检测	定期审查日志和告警

9.2 每用户隔离机制（重点）

每个用户获得独立的 Firecracker 微虚拟机（硬件级隔离，非容器命名空间）：

- **STS 作用域凭证**：用户只能访问自己 S3 命名空间下的文件（`dingtalk_<用户ID>/*`）
- **凭证环境变量隔离**：7 个 AWS 凭证环境变量从 OpenClaw 进程中剥离
- **零访问兜底**：如果 STS AssumeRole 失败，OpenClaw 以零 AWS 权限启动

9.3 Bedrock Guardrails 内容安全

策略类型	配置
内容过滤	仇恨、侮辱、色情、暴力、不当行为（HIGH 强度）+ 提示词注入攻击
话题拒绝	加密诈骗、钓鱼、自伤、武器制造、恶意软件、身份欺诈
PII 过滤	邮箱、电话（脱敏）；信用卡、AWS 密钥、密码（拦截）
词汇过滤	托管脏话列表 + 自定义敏感词（凭证路径、密钥标识）
自定义正则	

策略类型	配置
	AWS Access Key、AWS Secret Key、通用 API Key 模式匹配

模块 10：凭证轮换与密钥管理

10.1 系统凭证清单

密钥	位置	轮换频率建议
钉钉 AppKey/AppSecret	`openclaw/channels/dingtalk`	每 90 天或凭证泄露时
Gateway Token	`openclaw/gateway-token`	每 90 天
Webhook Secret	`openclaw/webhook-secret`	每 90 天
Cognito 密码密钥	`openclaw/cognito-password-secret`	每 180 天

10.2 轮换钉钉凭证

```
# 1. 在钉钉开放平台重新生成 AppSecret
# 2. 更新 Secrets Manager
aws secretsmanager update-secret \
  --secret-id openclaw/channels/dingtalk \
  --secret-string '{"clientId":"新AppKey","clientSecret":"新AppSecret"}' \
  --region us-west-2

# 3. 重启钉钉桥接服务
aws ecs update-service --cluster openclaw-dingtalk \
  --service openclaw-dingtalk-bridge --force-new-deployment \
  --region us-west-2
```

10.3 轮换系统密钥

```
# 轮换 Gateway Token（自动生成新 token）
NEW_TOKEN=$(openssl rand -hex 32)
aws secretsmanager update-secret \
  --secret-id openclaw/gateway-token \
  --secret-string "$NEW_TOKEN" \
  --region us-west-2

# 需要更新 Runtime 环境变量或终止现有会话使新 token 生效
agentcore stop-session -a openclaw_agent -s <SESSION_ID>
```

10.4 用户 API 密钥管理

用户通过 `api-keys` 技能自行管理第三方 API 密钥： - 存储在 Secrets Manager（`openclaw/user/<命名空间>/<密钥名>`） - KMS 加密、CloudTrail 审计 - 每用户最多 10 个密钥 - STS 作用域凭证确保用户只能访问自己的密钥

模块 11：定期安全巡检流程

11.1 每日巡检（5-10 分钟）

```
# 1. 检查 CloudWatch 告警状态
aws cloudwatch describe-alarms \
  --alarm-name-prefix "OpenClaw" \
  --state-value ALARM \
  --region us-west-2 \
  --query 'MetricAlarms[*].
{Name:AlarmName,State:StateValue,Reason:StateReason}'

# 2. 检查 Token 用量（过去 24 小时）
aws cloudwatch get-metric-statistics \
  --namespace OpenClaw/TokenUsage \
  --metric-name TotalTokens \
  --start-time $(date -u -d '24 hours ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 3600 --statistics Sum \
  --region us-west-2

# 3. 检查钉钉桥接服务健康状态
aws ecs describe-services --cluster openclaw-dingtalk \
```

```
  --services openclaw-dingtalk-bridge --region us-west-2 \
  --query 'services[0].
{running:runningCount,desired:desiredCount,events:events[:3]}'

# 4. 检查近期 Lambda 错误
aws logs filter-log-events \
  --log-group-name /openclaw/lambda/router --region us-west-2 \
  --start-time $(python3 -c "import time;
print(int((time.time()-86400)*1000))") \
  --filter-pattern "ERROR" \
  --query 'events | length(@)'
```

11.2 每周巡检（15-30 分钟）

检查项	操作	预期结果
Token 用量趋势	查看 `OpenClaw-Token-Analytics` 仪表板	无异常波动
Bedrock 延迟	查看 `OpenClaw-Operations` 仪表板 p99 延迟	< 10 秒
Guardrail 拦截率	CloudWatch Logs Insights 查询 guardrailAction	无异常增长
白名单审查	`./scripts/manage-allowlist.sh list`	只有授权用户
ECS 服务事件	AWS 控制台查看 ECS 服务事件	无频繁重启

```
# Guardrail 拦截查询（过去 7 天）
aws logs start-query \
  --log-group-name /aws/bedrock/invocation-logs \
  --start-time $(date -d '7 days ago' +%s) \
  --end-time $(date +%s) \
  --query-string 'stats count(*) by guardrailAction | sort count
desc' \
  --region us-west-2
```

11.3 每月巡检（1-2 小时）

检查项	操作	说明
ECR 镜像安全扫描	查看 ECR 扫描结果	关注 CRITICAL 和 HIGH 漏洞
凭证轮换检查	确认各密钥最后更新时间	超过 90 天的需要轮换
IAM 权限审计	检查执行角色权限是否过宽	遵循最小权限原则
成本审计	查看 AWS Cost Explorer	确认无异常费用增长
CDK 合规检查	`cdk synth` 运行 cdk-nag	确认无新增安全警告
CloudTrail 审查	检查敏感 API 调用	关注 Secrets Manager 访问

```
# ECR 镜像扫描结果
aws ecr describe-image-scan-findings \
  --repository-name bedrock-agentcore-openclaw_agent \
  --image-id imageTag=latest \
  --query 'imageScanFindings.findingSeverityCounts' \
  --region us-west-2

# 密钥最后更新时间
for secret in openclaw/channels/dingtalk openclaw/gateway-token
openclaw/webhook-secret; do
  echo "=== $secret ==="
  aws secretsmanager describe-secret --secret-id "$secret" --
region us-west-2 \
    --query
'{LastChanged:LastChangedDate,LastAccessed:LastAccessedDate}'
done

# CDK 合规检查（cdk-nag）
source .venv/bin/activate
cdk synth 2>&1 | grep -i "error\|warning"
```

11.4 季度安全评估（半天）

检查项	说明
红队测试	运行 `redteam/` 测试套件验证 Guardrails 有效性
依赖更新	检查 Node.js、Python 依赖是否有安全更新
架构审查	评估是否需要启用额外安全扩展（WAF、GuardDuty 等）
灾备验证	测试从 S3 恢复用户工作区
安全文档更新	更新安全操作手册

```
# 运行红队测试
cd redteam && npm install
AWS_REGION=us-west-2 npx promptfoo@latest eval --config
evalconfig.yaml
npx promptfoo@latest view  # 查看交互式报告
```

模块 12：预算与成本控制

12.1 主要成本来源

服务	计费方式	月估算（100 用户）
Bedrock 推理	按 token 计费	取决于使用量
Bedrock Guardrails	~$0.75/千次文本单元	按消息量增长
AgentCore Runtime	按会话时长计费	取决于活跃用户
ECS Fargate（钉钉桥接）	按 vCPU + 内存	~$30-50/月
NAT Gateway	按小时 + 流量	~$35-50/月
DynamoDB	按请求付费	通常 < $5/月
S3	按存储量 + 请求	通常 < $5/月
Secrets Manager	$0.40/密钥/月	~$5/月

12.2 预算告警配置

当前已配置的 CloudWatch 告警：

告警	阈值	动作
每日 Token 预算	> 1,000,000 tokens/小时	SNS 通知
每日成本预算	> $5 USD/小时	SNS 通知

告警	阈值	动作
Bedrock 限流	> 1 次/5 分钟	SNS 通知
Bedrock 延迟	p99 > 10 秒	SNS 通知

12.3 成本异常响应流程

```
# 1. 查找高用量用户
aws dynamodb query \
  --table-name openclaw-token-usage \
  --index-name GSI3 \
  --key-condition-expression "GSI3PK = :pk" \
  --expression-attribute-values "{\":pk\": {\"S\": \"DATE#$(date +%Y-%m-%d)\"}}" \
  --scan-index-forward false --limit 5 \
  --region us-west-2

# 2. 临时禁用高用量用户
./scripts/manage-allowlist.sh remove dingtalk:用户ID

# 3. 联系用户了解情况后，决定是否恢复访问
./scripts/manage-allowlist.sh add dingtalk:用户ID
```

模块 13：Guardrails 策略调整

13.1 查看当前 Guardrails 配置

```
# 获取 Guardrail ID
GUARDRAIL_ID=$(aws cloudformation describe-stacks \
  --stack-name OpenClawGuardrails \
  --query "Stacks[0].Outputs[?contains(OutputKey,'GuardrailId')].OutputValue" \
  --output text --region us-west-2)

# 查看详细配置
aws bedrock get-guardrail --guardrail-identifier $GUARDRAIL_ID --region us-west-2
```

13.2 调整策略

修改 `stacks/guardrails_stack.py` 中的配置，然后重新部署：

```
source .venv/bin/activate
cdk deploy OpenClawGuardrails --require-approval never
```

13.3 常见调整场景

场景	调整方式
误拦截正常业务内容	降低对应类别过滤强度（HIGH → MEDIUM）
需要拦截特定敏感词	在词汇过滤自定义列表中添加
需要脱敏新的 PII 类型	在 PII 过滤中添加实体类型
成本过高	设置 `enable_guardrails: false` 禁用（其他安全层仍有效）

13.4 验证策略变更

```
# 运行红队测试确认未降低安全水位
cd redteam && npx promptfoo@latest eval --config evalconfig.yaml
```

模块 14：应急响应预案

14.1 凭证泄露

```
# 1. 立即轮换泄露的凭证
aws secretsmanager update-secret \
  --secret-id openclaw/channels/dingtalk \
  --secret-string '{"clientId":"新凭证","clientSecret":"新密钥"}' \
  --region us-west-2

# 2. 重启服务加载新凭证
aws ecs update-service --cluster openclaw-dingtalk \
  --service openclaw-dingtalk-bridge --force-new-deployment \
  --region us-west-2

# 3. 检查 CloudTrail 确认是否有异常访问
```

```
aws logs start-query \
  --log-group-name /aws/cloudtrail \
  --start-time $(date -d '24 hours ago' +%s) \
  --end-time $(date +%s) \
  --query-string 'filter
eventSource="secretsmanager.amazonaws.com" | stats count(*) by
eventName, sourceIPAddress' \
  --region us-west-2
```

14.2 异常高用量

```
# 1. 识别异常用户
# 2. 临时从白名单移除
./scripts/manage-allowlist.sh remove dingtalk:可疑用户ID

# 3. 终止该用户会话
agentcore stop-session -a openclaw_agent -s <SESSION_ID>

# 4. 调查原因（查看 Bedrock invocation logs）
```

14.3 服务不可用

```
# 1. 检查各组件状态
aws ecs describe-services --cluster openclaw-dingtalk --services
openclaw-dingtalk-bridge --region us-west-2
agentcore status --agent openclaw_agent --verbose

# 2. 查看错误日志
aws logs tail /openclaw/dingtalk-bridge --since 30m --region us-
west-2

# 3. 尝试重启钉钉桥接
aws ecs update-service --cluster openclaw-dingtalk \
  --service openclaw-dingtalk-bridge --force-new-deployment --
region us-west-2
```

附录

A. 安全巡检清单（可打印）

频率	检查项	完成
每日	CloudWatch 告警无 ALARM 状态	☐
每日	Token 用量在正常范围	☐
每日	钉钉桥接服务运行中	☐
每周	白名单只包含授权用户	☐
每周	Guardrail 拦截率无异常	☐
每周	运维仪表板各指标正常	☐
每月	ECR 镜像扫描无 CRITICAL 漏洞	☐
每月	系统凭证在有效期内（< 90 天）	☐
每月	CDK 合规检查（cdk-nag）通过	☐
每月	成本在预算范围内	☐
季度	红队测试通过率 > 90%	☐
季度	依赖安全更新已应用	☐

B. 关键 AWS 资源清单

资源	名称/标识	用途
ECS 集群	`openclaw-dingtalk`	钉钉桥接服务
DynamoDB 表	`openclaw-identity`	用户身份、白名单
S3 存储桶	`openclaw-user-files-<ACCOUNT>-us-west-2`	用户文件
ECR 仓库	`bedrock-agentcore-openclaw_agent`	容器镜像
CloudWatch 仪表板	`OpenClaw-Operations`	运维监控
CloudWatch 仪表板	`OpenClaw-Token-Analytics`	Token 用量分析
Secrets Manager	`openclaw/channels/dingtalk`	钉钉凭证
Secrets Manager	`openclaw/gateway-token`	网关 Token

C. 常用命令速查表

```
# 用户管理
./scripts/manage-allowlist.sh add dingtalk:<ID>      # 添加用户
./scripts/manage-allowlist.sh remove dingtalk:<ID>    # 移除用户
./scripts/manage-allowlist.sh list                     # 查看白名单

# 服务状态
agentcore status --agent openclaw_agent --verbose     # Runtime 状态
agentcore invoke '{"action":"status"}' -a openclaw_agent  # 容器状态

# 日志
aws logs tail /openclaw/dingtalk-bridge --follow --region us-west-2

# 会话管理
agentcore stop-session -a openclaw_agent -s <SID>     # 终止会话

# 部署
./scripts/deploy.sh                                   # 全量部署
./scripts/deploy.sh --runtime-only                    # 仅更新 Runtime

# 安全
cdk synth 2>&1 | grep -i error                        # CDK 合规检查
```

D. 参考文档

文档	路径	说明
钉钉接入指南	`docs/dingtalk-setup-zh.md`	钉钉机器人创建与配置
安全架构	`docs/security.md`	完整安全架构（纵深防御、威胁模型）
Guardrails 运维手册	`docs/guardrails.md`	内容安全策略配置与监控
红队测试报告	`docs/redteam-audit.md`	对抗测试覆盖率与结果
系统架构	`docs/architecture.md`	整体架构图与数据流